

15

Functional Programming Languages

Türkçe Anlatım • Terimler İngilizce

Konu Anlatımı • **Lisp'e Özel Odak** • Scheme Tam Anlatım
ML • Haskell • F# • Dil Karşılaştırmaları
Review Questions (50) • Problem Set • Programming Exercises (22)

Gebze Technical University

CSE 341 — Programming Languages • Spring 2026
Sebesta, *Concepts of Programming Languages*, 12th Ed.

İçindekiler

1 Giriş ve Motivasyon	4
1.1 Neden Functional Programming?	4
1.2 Imperatif Dilin Temel Sorunu: State	4
2 Matematiksel Fonksiyonlar	5
2.1 Temel Tanımlar	5
2.2 Lambda Notation ve Lambda Calculus	5
2.3 Functional Forms (Higher-Order Fonksiyonlar)	5
3 Functional Programming Dillerinin Temelleri	6
3.1 Tasarım Hedefi	6
3.2 Trade-off Analizi	6
4 İlk Fonksiyonel Dil: Lisp	6
4.1 Motivasyon ve Tarihsel Bağlam	6
4.2 Veri Tipleri: Atoms ve Lists	6
4.3 Bağlantılı Liste Yapısı (Linked List)	7
4.4 CAR ve CDR: İsimlerin Kökeni	8
4.5 Cambridge Polish Notation ve S-Expressions	8
4.6 EVAL: Serendipitous Yorumlayıcı	8
4.7 Dynamic Scoping: Yanlışıklıkla Yapılan Seçim	9
5 Scheme: Kapsamlı Anlatım	9
5.1 REPL Modeli	9
5.2 Temel Aritmetik Fonksiyonlar	9
5.3 Fonksiyon Tanımlama: DEFINE	9
5.4 LAMBDA Expressions	10
5.5 Kontrol Akışı: IF ve COND	10
5.6 List Fonksiyonları: QUOTE, CAR, CDR, CONS	11
5.6.1 QUOTE: Neden Gerekli?	11
5.6.2 Predicate Fonksiyonlar: EQ?, EQV?, EQUAL?	12
5.6.3 LIST, APPEND ve Diğer List Operasyonları	12
5.7 Örnek Fonksiyonlar: Listenin Gerçek Gücü	13
5.7.1 member: Listede Arama	13
5.7.2 append: Özyinelemeli Liste Birleştirme	13
5.7.3 equalsimp ve equal: Liste Eşitliği	13
5.7.4 guess (intersection): Ortak Elemanlar	14
5.8 LET: Yerel Kapsam	14
5.9 Tail Recursion	15
5.10 Functional Forms: compose ve map	15
5.11 EVAL ile Dinamik Kod İnşası	16

6	Common Lisp	16
7	ML	17
7.1	Scheme'den Temel Farklar	17
7.2	Type Inference (Tip Çıkarımı)	17
7.3	Pattern Matching ile Fonksiyon Tanımlama	18
7.4	Currying ve Partial Evaluation	18
8	Haskell	19
8.1	Lazy Evaluation ve Sonsuz Listeler	20
9	F#	20
10	İmperatif Dillerde Functional Destek	21
11	Fonksiyonel ve İmperatif Dillerin Karşılaştırması	21
12	Bölüm Özeti: Tüm Kavramlar Bir Bakışta	22
13	Sınav İçin Kritik Noktalar	23
14	Review Questions — Soru Çözümleri	23
15	Problem Set — Soru Çözümleri	32
16	Programming Exercises — Çözümler	35

1 Giriş ve Motivasyon

1.1 Neden Functional Programming?

Programlama dillerinin büyük çoğunluğu *imperatif paradigmayı* benimser: bir değişkenin değeri değişir, program durumu adım adım ilerler, ve programcılık bilgisayara ne yapması gerektiğini adım adım tarif eder. Bu yaklaşım von Neumann mimarisinden doğrudan ilham almaktadır. Ancak bazı bilgisayar bilimcileri daha 1970'lerden itibaren şunu sorgulamıştır: *"Bilgisayara sırayla ne yapacağını söylemek zorunda mıyız?"*

Fortran'ın babası John Backus, 1978 yılında ACM Turing Ödülü konuşmasında şaşırtıcı bir argüman öne sürdü: *"Saf functional programlama dilleri, imperatif dillerden daha iyidir – çünkü ortaya çıkan programlar daha okunabilir, daha güvenilir ve doğruluğu kanıtlanabilir oluyor."* Bu konuşma fonksiyonel programlama araştırmalarını büyük ölçüde canlandırdı. Backus'un önerdiği FP dili yaygınlaşmasa da fikri çok etkili oldu.

Fonksiyonel programlamayı şöyle düşünebilirsiniz: Matematikte $f(x) = x^2 + 1$ yazdığınızda, f 'nin "hafızası" yoktur; aynı x için her zaman aynı sonucu verir. Fonksiyonel diller bu matematiksel saflığı programlamaya taşımaya çalışır. **State (durum)** kavramı ortadan kalkar, **değişken** kavramı dönüşür.

1.2 Imperatif Dilin Temel Sorunu: State

Büyük bir imperatif programda şunu takip etmek zorundasınızdır: şu anda her değişken ne değer taşıyor? Yüzlerce global ve yerel değişkenin anlık durumunu zihninde tutmak son derece zordur. Fonksiyonel dillerde ise değişken yoktur (en saf biçimde), dolayısıyla state de yoktur. Bu, hem test edilebilirliği artırır hem de eşzamanlı çalıştırmayı kolaylaştırır.

Temel Kavramlar

Referential Transparency (Referanssel Şeffaflık): Bir ifadenin değeri, yalnızca parametrelerine bağlıdır; çağrıldığı bağlama, programın durumuna veya dış değişkenlere bağlı değildir. Aynı girdi her zaman aynı çıktıyı üretir.

Pure Functional Language (Saf Fonksiyonel Dil): Değişken ve atama deyimi içermeyen; tüm hesaplamayı fonksiyon uygulamaları üzerinden yürüten dil.

Side Effect (Yan Etki): Bir fonksiyonun döndürdüğü değer dışında, programın geri kalanını etkileyen her türlü işlem (global değişken yazma, I/O, vs.).

Sınav sorusu: *"Pure functional language nedir, imperative language'dan ne farkı var?"* – Cevap üç eksenle kurulur: (1) değişken/state yok, (2) referential transparency var, (3) tekrar iteration değil recursion ile yapılır. **Haskell** tam anlamıyla saf fonksiyoneldir; **Scheme** ve **ML** ise fonksiyonel ağırlıklı olsa da bazı imperatif özellikler içerir.

2 Matematiksel Fonksiyonlar

2.1 Temel Tanımlar

Matematiksel Fonksiyon

Bir matematiksel fonksiyon, **domain** (tanım kümesi) adı verilen bir kümeden, **range** (değer kümesi) adı verilen başka bir kümeye yapılan bir *eşlemedir* (*mapping*). Parametreler domain'den seçilir; fonksiyonun ürettiği değer range'e aittir.

$$\text{cube}(x) \equiv x \times x \times x \quad \text{burada } x \in \mathbb{R}$$

Matematiksel fonksiyonların imperatif değişkenlerden temel farkı şudur: Fonksiyon değerlendirmesi sırasında parametre **sabit** kalır; değiştirilemez. Parametre, bir bellek konumunu değil, domain'den seçilmiş belirli bir değeri temsil eder.

2.2 Lambda Notation ve Lambda Calculus

Matematikçi Alonzo Church, fonksiyonu isimden bağımsız olarak tanımlamak için **lambda notasyonunu** geliştirdi. Lambda calculus, fonksiyon tanımı, fonksiyon uygulaması ve özyinelemeyi formal olarak ele alır. Bu sistem, fonksiyonel programlama dillerinin teorik temelidir. Turing makinesi ile eşdeğer hesaplama gücüne sahip olduğu kanıtlanmıştır.

Lambda Expression

Bir **lambda expression**, parametre listesini ve eşleme ifadesini belirten *isimsiz* bir fonksiyondur.

$$\lambda(x) x \times x \times x$$

Uygulama notasyonu: $(\lambda(x) x \times x \times x)(2)$ sonucu 8 üretir.

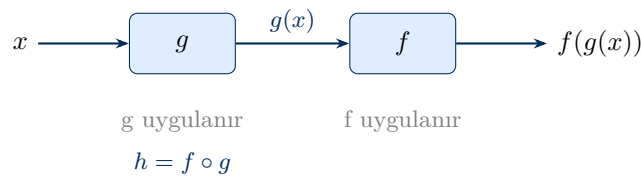
Lambda expression, fonksiyonun *kendisidir* – isme ihtiyaç yoktur.

2.3 Functional Forms (Higher-Order Fonksiyonlar)

Higher-Order Function / Functional Form

Higher-order function (fonksiyonel form), ya parametresi olarak bir veya daha fazla fonksiyon alan, ya da sonuç olarak bir fonksiyon üreten fonksiyondur.

- **Function Composition:** $h \equiv f \circ g$, yani $h(x) = f(g(x))$
- **Apply-to-All (α):** $\alpha(h, (2, 3, 4)) \rightarrow h$ her elemana uygulanır $\rightarrow (4, 9, 16)$



3 Functional Programming Dillerinin Temelleri

3.1 Tasarım Hedefi

Fonksiyonel bir dilin tasarım hedefi, matematiksel fonksiyonları mümkün olduğu kadar doğrudan programlamaya yansıtmaktır. Bu hedefe ulaşmak için şu sonuçlar kabul edilir:

- Değişken yoktur (en saf biçimde); dolayısıyla atama deyimi de yoktur.
- Tekrar (iteration) yerine **özyineleme (recursion)** kullanılır.
- Program = fonksiyon tanımları + fonksiyon uygulamaları.
- Yürütme = fonksiyon uygulamalarını değerlendirmek.

Saf Fonksiyonel Dilin Bileşenleri

1. **Primitive functions:** Dilin temel, önceden tanımlı fonksiyonları.
2. **Functional forms:** Fonksiyonlardan yeni fonksiyonlar inşa etme araçları.
3. **Function application:** Fonksiyonu parametrelere uygulama işlemi.
4. **Data structures:** Parametre ve sonuçları temsil etmek için yapılar.

3.2 Trade-off Analizi

Boyut	Fonksiyonel Diller	Imperatif Diller
Readability	Kısa, ifade gücü yüksek	Verbose, kontrol akışı detaylı
	Soyutlama kolay	Durum yönetimi gerekli
Reliability	Yan etki yok → test kolay	Yan etkiler hata kaynağı
	Genel 2× daha yavaş	Von Neumann'a optimize
Concurrency	Doğal (state yok)	Synchronization karmaşık

4 İlk Fonksiyonel Dil: Lisp

4.1 Motivasyon ve Tarihsel Bağlam

John McCarthy, 1959 yılında MIT'deki yapay zeka projesinde **Lisp**'i (LISt Processing) geliştirdi. Başlangıçta amacı yalnızca liste işleme için uygun bir notasyon üretmektir. İlk implementasyon IBM 704 makinesi üzerinde yapıldı. Lisp, hesaplamayı Turing makineleri yerine *sembolik liste işleme* ile modelleyen ilk dildir. Bu yaklaşım matematiksel fonksiyon teorisine çok daha yakındır. Bugün hâlâ kullanılan tüm LISP varyantlarının ortak atası bu 1959 tasarımıdır.

4.2 Veri Tipleri: Atoms ve Lists

Orijinal Lisp'te yalnızca **iki** veri kategorisi vardı:

Orijinal Lisp Veri Tipleri

- **Atom:** Sembol (identifier) veya sayısal literal. Örnek: A, JAKE, 42.
- **List:** Elemanları parantezle sınırlandırılmış, birbirine bağlı veri yapısı. Örnek: (A B C D), (A (B C) D).

Atomlar ve listeler, imperatif dillerdeki anlamda *tipler* değildir; orijinal Lisp **tipsiz** (typeless) bir dildir.

Listing 1: Lisp list örnekleri

```

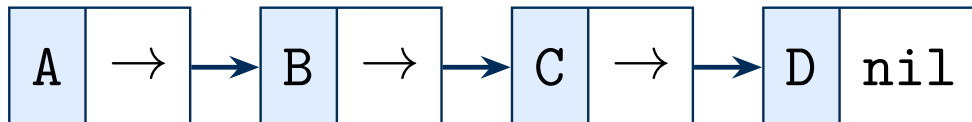
1 ; Simple list - düz liste
2 (A B C D)
3
4 ; Nested list - iç içe liste
5 (A (B C) D (E (F G)))
6 ; Bu liste 4 eleman içerir:
7 ;   1. atom A
8 ;   2. alt-liste (B C)
9 ;   3. atom D
10 ;   4. alt-liste (E (F G))

```

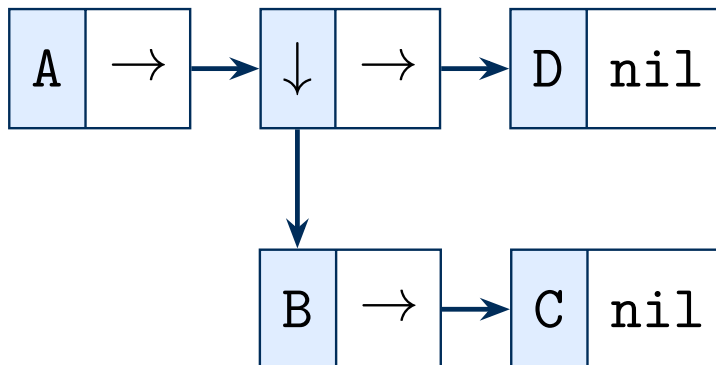
4.3 Bağlantılı Liste Yapısı (Linked List)

Bellekte bir Lisp listesi, her bir düğümün iki pointer (gösterici) içerdiği bir **linked list** olarak tutulur. Birinci pointer veriyi (ya da alt listeyi) gösterir, ikinci pointer ise listedeki bir sonraki elemanı.

(A B C D)



(A (B C) D)



4.4 CAR ve CDR: İsimlerin Kökeni

Lisp'in ilk implementasyonunda kullanılan **IBM 704** bilgisayarının bellek sözcükleri iki alandan oluşuyordu: *address* (adres) ve *decrement* (azaltma). Makine ayrıca bu alanları okuyan iki komut içeriyordu:

- **CAR:** *Contents of the Address part of a Register* – bir kayıttaki adres bölümünün içeriği.
- **CDR:** *Contents of the Decrement part of a Register* – bir kayıttaki azaltma bölümünün içeriği.

Bir liste düğümü bu iki alana mükemmel sığıyordu: *address* → veri işaretçisi, *decrement* → sonraki düğüm işaretçisi. İsimler tuhaf görünse de çok pratik bir mühendislik kararının ürünüdür.

CAR, CDR, CONS

- **CAR:** Listenin *ilk elemanını* (başını) döndürür. ML'deki `hd`'ye karşılık gelir.
- **CDR:** Listenin *geri kalanını* (kuyruğunu) döndürür. ML'deki `tl`'ye karşılık gelir.
- **CONS:** Bir eleman ile bir listeyi birleştirerek yeni bir liste üretir. ML'deki `::`'e karşılık gelir.

4.5 Cambridge Polish Notation ve S-Expressions

Lisp'te fonksiyon çağrısı **prefix** (önek) biçimindedir – *Cambridge Polish* adıyla anılır:

Listing 2: Cambridge Polish prefix notasyonu

```
1 ; (function_name param1 param2 ...)
2 (+ 5 7) ; => 12
3 (+ 3 4 7 6) ; => 20 (toplama çok parametrelili)
4 (* 3 (+ 2 4)) ; => 18 (ic ice ifadeler)
```

S-expression (symbolic expression): Hem Lisp kodu hem de veri aynı sözdizimini – iç içe parantezli listeler – kullanır. Bu, kod-veri birliğinin temelidir.

4.6 EVAL: Serendipitous Yorumlayıcı

McCarthy, orijinalinde bir *evrensel Lisp fonksiyonu* yazmak istiyordu – tüm Lisp fonksiyonlarını değerlendiren bir **EVAL** fonksiyonu. Bu saf teorik bir egzersizdi. Ancak AI projesindeki mühendisler **Stephen B. Russell** ve **Daniel J. Edwards**, EVAL'in aslında doğrudan bir *Lisp yorumlayıcısı* olarak gerçekleştirilebileceğini fark etti ve bunu hemen yaptı. Sonuç: Planlanan M-notation hiç tamamlanmadı; S-expression hem kod hem veri için tek notasyon hâline geldi.

Bu "kaza" üç kalıcı sonuç doğurdu:

1. Tüm erken Lisp implementasyonları yorumlayıcıydı.
2. Kod ve veri aynı formu paylaştığından, programlar başka programlar üretebilir.
3. Dil tasarımının büyük bölümü donduruldu – bazı garip özellikler de bunun kurbanı oldu.

4.7 Dynamic Scoping: Yanlılıkla Yapılan Seçim

1975 öncesi Lisp'lerin neredeyse tamamı **dynamic scoping** kullanıyordu. Yani bir fonksiyon, tanımlandığı ortamda değil, *çağrıldığı ortamda* değerlendirilen değişkenlere bakıyordu. Bunun bilinçli bir karar olmadığı; scoping konusunun yeterince düşünülmediği için bu yolun seçildiği sanılmaktadır. Çağdaş Lisp varyantları (özellikle Scheme) static scoping'e geçmiştir.

5 Scheme: Kapsamlı Anlatım

Scheme, 1975 yılında MIT'de **Sussman** ve **Steele** tarafından geliştirilmiştir. Lisp'in bir lehçesi olan Scheme, üç temel özelliğiyle öne çıkar:

1. **Küçük ve sade:** Tüm dil az sayıda temel yapıdan oluşur.
2. **Static (lexical) scoping:** Değişken kapsamı, programın metinsel yapısıyla belirlenir.
3. **First-class functions:** Fonksiyonlar değer gibi davranır; parametre geçilebilir, liste elemanı yapılabilir, döndürülebilir.

5.1 REPL Modeli

REPL – Read-Evaluate-Print Loop

Scheme yorumlayıcısı sonsuz bir döngüde çalışır: kullanıcıdan bir ifade **okur (read)**, o ifadeyi **değerlendirir (evaluate)**, sonucu **yazar (print)**, ve tekrar başa döner. Python ve Ruby yorumlayıcıları da bu modeli kullanır. Yorumlama çekirdeği EVAL fonksiyonudur.

5.2 Temel Aritmetik Fonksiyonlar

Listing 3: Scheme temel aritmetik

```

1 42                ; => 42  (literal kendini değerlendirir)
2 (* 3 7)           ; => 21
3 (+ 5 7 8)         ; => 20  (+ sifir veya daha fazla parametre alır)
4 (- 5 6)           ; => -1
5 (- 15 7 2)        ; => 6   (ilkten diğerleri çıkarılır)
6 (- 24 (* 4 3))    ; => 12  (ic ice ifade)
7 (sqrt 25)         ; => 5
8 (max 3 7 2)       ; => 7

```

5.3 Fonksiyon Tanımlama: DEFINE

DEFINE: İki Kullanım Biçimi

Biçim 1 – İsme değer bağlama:

```
(DEFINE pi 3.14159)
(DEFINE two-pi (* 2 pi))
```

Bu, Java'daki final float PI = 3.14159; ile eşdeğerdir. Yeniden atama yapılamaz.

Biçim 2 – İsme lambda bağlama (fonksiyon tanımlama):

```
(DEFINE (fonksiyon-adı parametreler) (ifade))
```

LAMBDA kelimesi kaldırılmış kısaltılmış biçimdir.

DEFINE, normal bir primitive fonksiyon değildir; özel biçimde (special form olarak) değerlendirilir. Normal bir fonksiyon olsaydı, (DEFINE x 10) çağrısında EVAL önce x'i değerlendirirdi – ama x henüz tanımlı değildir, hata olurdu. DEFINE bu yüzden EVAL'in özel muamele ettiği bir formdur: parametrelerini *değerlendirmeden* önce işler.

Ayrıca: DEFINE ile oluşturulan isimler **değişken değil, sabit değer isimleridir**. Bir kez bağlandıktan sonra yeniden bağlanamazlar (teknik olarak redefine edilebilirler ama bu iyi uygulama değildir).

Listing 4: DEFINE kullanım örnekleri

```
1 ; Sabit deger tanimlama
2 (DEFINE pi 3.14159)
3 (DEFINE two-pi (* 2 pi)) ; two-pi = 6.28318
4
5 ; Fonksiyon tanimlama - kare alma
6 (DEFINE (square n) (* n n))
7 (square 5) ; => 25
8
9 ; Fonksiyon tanimlama - hipotenus
10 (DEFINE (hypotenuse side1 side2)
11   (sqrt (+ (square side1) (square side2))))
12 (hypotenuse 3 4) ; => 5
```

5.4 LAMBDA Expressions

Listing 5: LAMBDA ile isimsiz fonksiyon

```
1 ; Lambda expression - isimsiz fonksiyon
2 (LAMBDA (x) (* x x)) ; x^2 hesaplayan isimsiz fonksiyon
3
4 ; Uygulama: parantez icinde lambda, sonra arguman
5 ((LAMBDA (x) (* x x)) 7) ; => 49
6
7 ; Çok parametrelili lambda
8 (LAMBDA (a b c x)
9   (+ (* a x x) (* b x) c)) ; ax^2 + bx + c
```

5.5 Kontrol Akışı: IF ve COND

Listing 6: IF ve COND kontrol yapıları

```
1 ;; IF - uc parametrelili: predicate, then, else
2 (DEFINE (factorial n)
3   (IF (<= n 1)
4     1
5     (* n (factorial (- n 1)))))
6
7 (factorial 5) ; => 120
```

```

8
9 ;; COND - cogul secim (case/switch gibi)
10 (DEFINE (leap? year)
11   (COND
12     ((ZERO? (MODULO year 400)) #T) ; 400'e bolunuyorsa artik yıl
13     ((ZERO? (MODULO year 100)) #F) ; 100'e bolunuyorsa degildir
14     (ELSE (ZERO? (MODULO year 4)))) ; 4'e bolunuyorsa artik yıl
15
16 (leap? 2024) ; => #T
17 (leap? 1900) ; => #F

```

5.6 List Fonksiyonları: QUOTE, CAR, CDR, CONS

5.6.1 QUOTE: Neden Gerekli?

Scheme'de *her şey* değerlendirilir (EVAL tarafından). Bir liste yazdığınızda yorumlayıcı onu *fonksiyon çağırısı* olarak okur. Ama bazen listeyi **veri** olarak, hesaplanmaması gereken bir nesne olarak kullanmak istersiniz. QUOTE tam bunu sağlar: parametresini değerlendirmeden geri döndürür. Scheme'de kodla veri aynı forma sahip olduğundan bu ayırım zorunludur. '(A B C) kısaltması (QUOTE (A B C)) ile tamamen eşdeğerdir.

Listing 7: CAR, CDR, CONS örnekleri

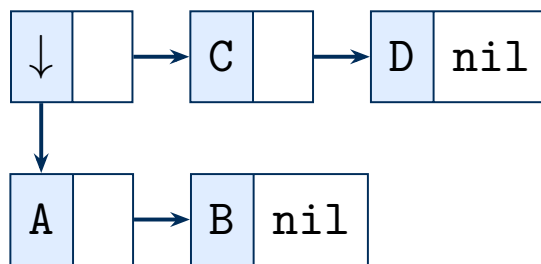
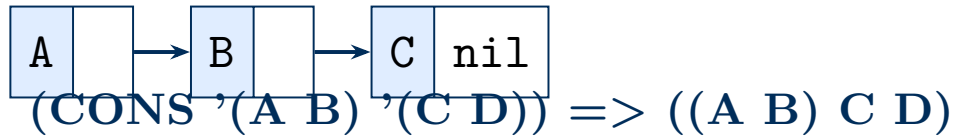
```

1 ;; CAR - ilk eleman
2 (CAR '(A B C)) ; => A
3 (CAR '((A B) C D)) ; => (A B) [alt liste!]
4 (CAR '(A)) ; => A
5 ; (CAR '()) => HATA!
6
7 ;; CDR - geri kalan (kuyruk)
8 (CDR '(A B C)) ; => (B C)
9 (CDR '((A B) C D)) ; => (C D)
10 (CDR '(A)) ; => ()
11 ; (CDR '()) => HATA!
12
13 ;; CONS - eleman + liste -> yeni liste
14 (CONS 'A '()) ; => (A)
15 (CONS 'A '(B C)) ; => (A B C)
16 (CONS '() '(A B)) ; => () A B
17 (CONS '(A B) '(C D)) ; => ((A B) C D)
18
19 ;; Not: CONS (CAR lst) (CDR lst) == lst
20 (CONS (CAR '(A B C)) (CDR '(A B C))) ; => (A B C)

```

$(\text{CONS } 'A \text{ } '(B \ C)) \Rightarrow (A \ B \ C)$

yeni dugum



5.6.2 Predicate Fonksiyonlar: EQ?, EQV?, EQUAL?

Bu üçü farklı koşullarda çalışır ve sınav sorularında çok sık karıştırılır:

Fonksiyon	Çalıştığı Durum	Notlar
EQ?	Sembolik atomlar için pointer eşitliği	Sayılar ve listeler için güvenilirmez
EQV?	Sembolik + nümerik atomlar, değer karşılaştırması	Listeler için hâlâ güvenilirmez
EQUAL?	Her tür için yapısal eşitlik	En yavaşı; yalnızca gerektiğinde kullan
=	Yalnızca sayılar	Semboller için hata verir

$(\text{EQ? } '(A \ B) \ '(A \ B)) \rightarrow$ sonuç belirsiz (pointer aynı mı değil mi bilinmez). $(\text{EQUAL? } '(A \ B) \ '(A \ B)) \rightarrow$ her zaman #T.

5.6.3 LIST, APPEND ve Diğer List Operasyonları

Listing 8: LIST, APPEND, birlesik CAR/CDR

```

1 ;; LIST - birden fazla elemandan liste olusturur
2 (LIST 'apple 'orange 'grape) ; => (apple orange grape)
3 ; LIST, ic ice CONS'a esittir:
4 ; (CONS 'apple (CONS 'orange (CONS 'grape '())))
5
6 ;; Bilesik CAR/CDR - 4 harfe kadar
7 (CADR '(A B C)) ; = (CAR (CDR '(A B C))) => B
8 (CADDR '(A B C)) ; = (CAR (CDR (CDR '(A B C)))) => C
9 (CAAR '((A B) C)) ; = (CAR (CAR '((A B) C))) => A

```

```

10
11 ;; APPEND - iki listeyi birbirine ekler
12 (append '(A B) '(C D R))           ; => (A B C D R)
13 (append '((A B) C) '(D (E F)))     ; => ((A B) C D (E F))

```

5.7 Örnek Fonksiyonlar: Listenin Gerçek Gücü

5.7.1 member: Listede Arama

Listing 9: member – listede eleman arama

```

1 ;; Bir atomun basit listede olup olmadigini kontrol eder
2 (DEFINE (member atm a-list)
3   (COND
4     ((NULL? a-list) #F) ; bos liste -> bulunamadi
5     ((EQ? atm (CAR a-list)) #T) ; basta bulundu
6     (ELSE (member atm (CDR a-list)))) ; geri kalanı ara
7
8 (member 'B '(A B C)) ; => #T
9 (member 'B '(A C D E)) ; => #F
10
11 ;; DIKKAT: NULL? testi CAR'dan önce gelmeli!
12 ;; (CAR '()) HATA verir.

```

5.7.2 append: Özyinelemeli Liste Birleştirme

Listing 10: append – iki listeyi birleştirme

```

1 (DEFINE (append list1 list2)
2   (COND
3     ((NULL? list1) list2) ; baz durum
4     (ELSE (CONS (CAR list1) ; ilk elemanı al
5                 (append (CDR list1) list2)))) ; gerisi ile
6                 tekrarla
7
8 ;; Izi (trace):
9 ;; (append '(A B) '(C D))
10 ;;   CONS A (append '(B) '(C D))
11 ;;   CONS B (append '() '(C D))
12 ;;   => '(C D)
13 ;;   => '(B C D)
14 ;;   => '(A B C D)

```

5.7.3 equalsimp ve equal: Liste Eşitliği

Listing 11: equalsimp – basit liste karşılaştırma

```

1 (DEFINE (equalsimp list1 list2)
2   (COND
3     ((NULL? list1) (NULL? list2)) ; her ikisi bitmiş mi?
4     ((NULL? list2) #F) ; list2 kısa
5     ((EQ? (CAR list1) (CAR list2)) ; başlar eşit mi?
6      (equalsimp (CDR list1) (CDR list2)))
7     (ELSE #F)))
8
9 ;; Genel liste eşitliği (alt listeler dahil):

```

```

10 (DEFINE (equal list1 list2)
11   (COND
12     ((NOT (LIST? list1)) (EQ? list1 list2)) ; atom karsilastirma
13     ((NOT (LIST? list2)) #F)
14     ((NULL? list1) (NULL? list2))
15     ((NULL? list2) #F)
16     ((equal (CAR list1) (CAR list2)) ; ozineleme ile CAR
17      (equal (CDR list1) (CDR list2)))
18     (ELSE #F)))

```

5.7.4 guess (intersection): Ortak Elemanlar

Listing 12: guess – iki kumenin kesisimi

```

1 (DEFINE (guess list1 list2)
2   (COND
3     ((NULL? list1) '())
4     ((member (CAR list1) list2) ; list1'in basi list2'
5      de mi?
6      (CONS (CAR list1) (guess (CDR list1) list2))) ; evet -> dahil
7      et
8      (ELSE (guess (CDR list1) list2))) ; hayir -> atla
9   (guess '(A B C D) '(B D E)) ; => (B D) -- kesiSim bulundu

```

5.8 LET: Yerel Kapsam

LET

LET, isimleri geçici olarak değerlere bağlar ve bu bağlamaları bir ifadenin değerlendirilmesinde kullanır. Bu isimler daha sonra yeni değerlere yeniden bağlanamaz.

LET aslında bir LAMBDA uygulamasının kısaltmasıdır:

```
(LET ((alpha 7)) (* 5 alpha))
```

≡

```
((LAMBDA (alpha) (* 5 alpha)) 7)
```

Listing 13: LET ile ikinci dereceden denklem kokleri

```

1 ;; ax^2 + bx + c = 0 denkleminin gercek kokleri
2 (DEFINE (quadratic-roots a b c)
3   (LET (
4     (root-part-over-2a
5       (/ (SQRT (- (* b b) (* 4 a c))) (* 2 a)))
6     (minus-b-over-2a (/ (- 0 b) (* 2 a)))
7   )
8   (LIST (+ minus-b-over-2a root-part-over-2a)
9         (- minus-b-over-2a root-part-over-2a)))
10
11 (quadratic-roots 1 -5 6) ; => (3 2) kokler: x=3, x=2

```

5.9 Tail Recursion

Tail Recursion (Kuyruk Özyinelemesi)

Bir fonksiyon, özyinelemeli çağrısı *fonksiyondaki son işlem* ise **tail recursive** (kuyruk özyinelemeli) dir. Kuyruk özyinelemeli fonksiyonlar, stack büyütmeden iteration'a dönüştürülebilir – bu da büyük verimliliği artırır.

Scheme standardı, tüm tail recursive fonksiyonların iteration'a çevrilmesini **zorunlu** kılar.

Listing 14: Tail recursion donusumu: factorial

```

1 ;; Tail-recursive OLMAYAN versiyon (son islem carpma)
2 (DEFINE (factorial n)
3   (IF (<= n 1)
4     1
5     (* n (factorial (- n 1))))) ; son islem: carp
6
7 ;; n=5 icin stack durumu:
8 ;; fact(5) * fact(4) * fact(3) * fact(2) * fact(1)
9 ;; -- n adet activation record gerekirNot!
10
11 ;; Tail-recursive versiyon (helper ile)
12 (DEFINE (facthelper n partial)
13   (IF (<= n 1)
14     partial
15     (facthelper (- n 1) (* n partial)))) ; son islem: ozineleme
16
17 (DEFINE (factorial n) (facthelper n 1))
18
19 ;; n=5 icin: facthelper(4,5) -> facthelper(3,20)
20 ;; -> facthelper(2,60) -> facthelper(1,120) -> 120
21 ;; Tek activation record yeter!
```

Tail-Recursive OLMAYAN

fact(1)=1

fact(2): 2*...

fact(3): 3*...

fact(4): 4*...

fact(5): 5*...

5 activation record

Tail-Recursive

facthelper(1,120)

(onceki silinir)

facthelper(2,60)

(onceki silinir)

facthelper(5,1)

Tek frame yeterli!

5.10 Functional Forms: compose ve map

Listing 15: compose ve map implementasyonu

```

1 ;; COMPOSE: iki fonksiyonu birlestir
2 (DEFINE (compose f g)
3   (LAMBDA (x) (f (g x))))
4
5 ;; Ornek kullanim:
6 ((compose CAR CDR) '((a b) c d)) ; CADR => c
```

```

7 ((compose CDR CAR) '((a b) c d)) ; CDAR => (b)
8
9 ;; Ucuncu elemani almak (CADDR alternatifi):
10 (DEFINE (third lst)
11   ((compose CAR (compose CDR CDR)) lst))
12
13 ;; MAP: fonksiyonu listedeki her elemana uygula
14 (DEFINE (map fun a-list)
15   (COND
16     ((NULL? a-list) '())
17     (ELSE (CONS (fun (CAR a-list))
18                  (map fun (CDR a-list))))))
19
20 ;; Ornek: kupler listesi
21 (map (LAMBDA (num) (* num num num)) '(3 4 2 6))
22 ;; => (27 64 8 216)
23
24 ;; Lambda burada anonimdir - ismiz fonksiyon hemen kullanilir

```

5.11 EVAL ile Dinamik Kod İnşası

Listing 16: EVAL ile sayi listesini toplama

```

1 ;; Versiyon 1: ozinelemeli toplama
2 (DEFINE (adder a-list)
3   (COND
4     ((NULL? a-list) 0)
5     (ELSE (+ (CAR a-list) (adder (CDR a-list))))))
6
7 ;; Versiyon 2: EVAL ile dinamik ifade olusturma
8 (DEFINE (adder-eval a-list)
9   (COND
10     ((NULL? a-list) 0)
11     (ELSE (EVAL (CONS '+ a-list)))))
12 ;; (adder-eval '(3 4 5))
13 ;; => (EVAL (CONS '+ '(3 4 5)))
14 ;; => (EVAL '(+ 3 4 5))
15 ;; => 12
16
17 ;; NOT: + teklif almalidir ('+ seklinde)
18 ;; yoksa EVAL onu degerlendirir

```

Scheme'de (ve Lisp'te) programlar ve veriler aynı formata sahip olduğundan, bir program çalışırken başka bir program inşa edebilir ve onu EVAL ile hemen çalıştırabilir. Bu, meta-programlama ve makro sistemlerinin temelidir – ve Lisp'i diğer dillerden kökten farklı kılan özelliktir.

6 Common Lisp

Common Lisp (1984), 1980'lerin başındaki çeşitli Lisp lehçelerini (Scheme dahil) tek bir çatı altında toplamak için geliştirildi. Büyük ve karmaşık bir dil olsa da temeli orijinal Lisp'tir.

Boyut	Scheme	Common Lisp
Boyut	Küçük, sade	Büyük, karmaşık
Scoping	Yalnızca static	Static (varsayılan) + dynamic (special)
Amaç	Eğitim, araştırma	Ticari AI uygulamaları
Tipler	Tip çıkarımı yok	Zengin tip sistemi, diziler, kayıtlar
Makrolar	Sınırlı	Güçlü makro sistemi (DEFMACRO)
OOP	Yok	CLOS (Common Lisp Object System)

Listing 17: Common Lisp: factorial ve DOLIST

```

1 ;; Factorial (DEFUN ile tanımlama - Scheme'deki DEFINE gibi)
2 (DEFUN factorial (n)
3   (IF (<= n 1)
4       1
5       (* n (factorial (- n 1)))))
6
7 ;; DOLIST - iteratif liste isleme makrosu
8 (DOLIST (x '(1 2 3)) (print x))
9 ;; => 1 2 3 NIL
10
11 ;; Backquote ve comma
12 '(a (* 3 4) c)      ; => (a (* 3 4) c) -- hesaplanmaz
13 '(a ,( * 3 4) c)    ; => (a 12 c)      -- comma ile hesaplanır

```

7 ML

ML (Meta Language), 1973 yılında Edinburgh Üniversitesi'nde Robin Milner tarafından geliştirildi. Başlangıçta bir teorem ispat sisteminin komut dili olarak tasarlandı. Güçlü tiplendirme ve tip çıkarımı sistemleri, daha sonra OCaml, Haskell ve F# gibi dillere ilham verdi.

7.1 Scheme'den Temel Farklar

ML, Scheme'den iki temel açıdan ayrılır: (1) **güçlü tip sistemi** ve (2) **imperatif dil sözdizimine benzer sözdizim**. Scheme'de yazdığımız `(+ 3 4)` yerine ML'de `3 + 4` yazılır.

7.2 Type Inference (Tip Çıkarımı)

Type Inference

Değişken veya parametre türlerinin açıkça belirtilmesine gerek kalmadan, derleyicinin ifadedeki bağlamdan türleri otomatik olarak *çıkarımasıdır*. ML, OCaml, Haskell ve F# bu özelliği destekler.

Listing 18: ML'de tip çıkarımı

```

1 (* Tip çıkarımı: 3.14159 real turu -> r de real *)
2 fun circumf(r) = 3.14159 * r * r;
3 (* circumf : real -> real *)
4

```

```

5  (* 10 int olduğundan x de int *)
6  fun times10(x) = 10 * x;
7  (* times10 : int -> int *)
8
9  (* * operatorundan int cikarimi yapilir *)
10 fun square(x) = x * x;
11 (* square : int -> int *)
12
13 (* Acik real donusturme *)
14 fun square(x : real) = x * x;
15 (* square : real -> real *)

```

7.3 Pattern Matching ile Fonksiyon Tanımlama

Listing 19: ML'de pattern matching ile factorial

```

1  (* IF ile yazim *)
2  fun fact(n : int) : int =
3    if n <= 1 then 1
4    else n * fact(n - 1);
5
6  (* Pattern matching ile (daha kisa, daha net) *)
7  fun fact(0) = 1
8    | fact(1) = 1
9    | fact(n : int) : int = n * fact(n - 1);
10
11 (* Liste uzunlugu - h::t kalibi *)
12 fun length([]) = 0
13    | length(h :: t) = 1 + length(t);
14
15 (* Append - liste birlestirme *)
16 fun append([], lis2) = lis2
17    | append(h :: t, lis2) = h :: append(t, lis2);

```

7.4 Currying ve Partial Evaluation

Currying ve Partial Evaluation

Currying: Birden fazla parametrelili bir fonksiyonu, tek parametrelili ve geriye fonksiyon döndüren bir fonksiyon zincirine dönüştürme.

Partial Evaluation: Curried bir fonksiyona parametre sayısından daha az argüman verilerek yeni, özelleşmiş fonksiyonlar üretme.

Listing 20: ML'de currying ve partial evaluation

```

1  (* Curried fonksiyon - virgul yok! *)
2  fun add a b = a + b;
3  (* add : int -> int -> int *)
4
5  (* Normal cagri *)
6  add 3 5;  (* => 8 *)
7
8  (* Partial evaluation - add5 yeni bir fonksiyon *)
9  fun add5 x = add 5 x;
10 add5 10;  (* => 15 *)
11
12 (* filter - predicate ile liste filtreleme *)

```

```

13 filter (fn(x) => x < 100) [25, 1, 50, 711, 100, 150, 27];
14 (* => [25, 1, 50, 27] *)
15
16 (* map ve fonksiyon kompozisyonu o operatoru *)
17 fun cube x = x * x * x;
18 val newList = map cube [1, 3, 5];
19 (* => [1, 27, 125] *)
20
21 val h = g o f; (* h(x) = g(f(x)) *)

```

- ML tip dönüştürmesi yapmaz: `square(2.75)` – `square`'in parametresi `int` ise – **hata** verir.
- ML fonksiyon aşırı yüklemeye izin vermez: `int` ve `real` için ayrı `square` tanımlayamazsınız.
- `val` bir atama deyimi **değildir**: `val x = 5`; bağlama yapar; `x`'i ilerleyen kod bloklarında tekrar `val x = 10`; ile *yeni bir bağlama* oluşturabilirsiniz ama bu önceki `x`'i değiştirmez.

8 Haskell

Haskell (1990), ML ile benzer sözdizimi ve tür sistemine sahip olmakla birlikte üç temel özelliğiyle ayrışır:

Özellik	Haskell	ML
Evaluation	Lazy (tembel)	Strict (katı)
Function overloading	Var (type classes)	Yok
Saflık (Purity)	Saf fonksiyonel	Bazı yan etkiler mevcut
Tip sistemi	Polymorphic	Monomorphic varsayılan

Listing 21: Haskell: guards ve list comprehension

```

1  -- Pattern matching ile factorial
2  fact 0 = 1
3  fact 1 = 1
4  fact n = n * fact (n - 1)
5
6  -- Guards ile daha net yazım
7  fact n
8    | n == 0    = 1
9    | n == 1    = 1
10   | n > 1      = n * fact(n - 1)
11
12 -- List comprehension: 1'den 50'ye kup sayilari
13 cubes = [n * n * n | n <- [1..50]]
14
15 -- Quicksort - son derece kısa ve elegant!
16 sort [] = []
17 sort (h:t) = sort [b | b <- t, b <= h]
18             ++ [h]
19             ++ sort [b | b <- t, b > h]
20
21 -- Bir sayinin carpanlarini bul
22 factors n = [i | i <- [1..n 'div' 2], n 'mod' i == 0]

```

8.1 Lazy Evaluation ve Sonsuz Listeler

Lazy Evaluation ve Strict Evaluation

Strict language: Bir fonksiyon çağrılmadan önce tüm argümanlar tam olarak değerlendirilir (Scheme, ML).

Lazy (nonstrict) language: Bir ifade yalnızca değerine gerçekten ihtiyaç duyulduğunda değerlendirilir. Bu, sonsuz veri yapılarına olanak tanır.

Listing 22: Sonsuz listeler ve lazy evaluation

```

1  -- Sonsuz listeler (lazy evaluation ile calisir)
2  positives = [0..]           -- 0,1,2,3,...
3  evens = [2, 4..]           -- 2,4,6,8,...
4  squares = [n * n | n <- [0..]] -- 0,1,4,9,16,...
5
6  -- Belirli bir sayi tam kare mi? (squares sonsuz ama lazily
   hesaplanir)
7  member2 n (m:x)
8    | m < n      = member2 n x
9    | m == n     = True
10   | otherwise  = False
11
12 -- member2 16 squares => True (4. elemanda durur)

```

9 F#

F# (2005), Microsoft'un .NET platformu için geliştirilen, OCaml temelli fonksiyonel bir dildir. Fonksiyonel programlamayı imperatif ve nesne yönelimli programlama ile birleştirir.

Listing 23: F# temel örnekler

```

1  // Temel deger baglama
2  let pi = 3.14159;;
3  let twoPi = 2.0 * pi;; (* NOT: 2 degil 2.0 - tur donusamaz! *)
4
5  // Ozineleme - rec anahtar sozcugu zorunlu
6  let rec factorial x =
7    if x <= 1 then 1
8    else x * factorial(x - 1);;
9
10 // Pipeline operatoru |>
11 let myNums = [1; 2; 3; 4; 5]
12 let evenTimesFive =
13   myNums
14   |> List.filter (fun n -> n % 2 = 0)
15   |> List.map    (fun n -> 5 * n)
16 // => [10; 20]
17
18 // Fonksiyon kompozisyonu >>
19 let addOne x = x + 1
20 let double x = x * 2
21 let addOneThenDouble = addOne >> double
22 addOneThenDouble 3 // => (3+1)*2 = 8
23
24 // Sequence (tembel) vs List (acgozlu)
25 let lazySeq = seq {0..100000000} // sadece ihtiyac duyulunca
   uretilir

```

```
26 let eagerList = [0..100]           // tumu hemen bellekte
```

- **rec** anahtar sözcüğü unutulursa özyinelemeli fonksiyon **derlenmez**.
- F# sayısal dönüşüm yapmaz: `2 * pi` hata verir; `2.0 * pi` doğrudur.
- F# liste elemanları **noktalı virgülle** ayrılır: `[1; 2; 3]` – virgülle değil.
- Pipeline `|>` operatörü: sol taraftaki değeri sağdaki fonksiyona *son argüman* olarak gönderir.

10 İmperatif Dillerde Functional Destek

Dil	Lambda sözdizimi	map/filter	Currying	Closure
JavaScript	<code>(x) => expr</code>	<code>Array.map</code>	manuel	Evet
C#	<code>x => expr</code>	LINQ	<code>Func<></code>	Evet
Java 8+	<code>x -> expr</code>	Stream	manuel	Evet
Python	<code>lambda x: expr</code>	<code>map()</code> , <code>filter()</code>	<code>partial()</code>	Evet
Ruby	<code>lambda { x ...}</code>	<code>.map</code>	<code>.curry</code>	Evet

Listing 24: Python'da functional özellikler

```
1 # Python lambda expressions
2 square = lambda x: x ** 2
3 square(5) # => 25
4
5 # map ve filter
6 result = list(map(lambda x: x ** 3, [2, 4, 6, 8]))
7 # => [8, 64, 216, 512]
8
9 # Partial application
10 from functools import partial
11 add = lambda a, b: a + b
12 add5 = partial(add, 5)
13 add5(15) # => 20
```

11 Fonksiyonel ve İmperatif Dillerin Karşılaştırması

Boyut	Fonksiyonel	İmperatif
Syntax karmaşıklığı	Basit (Lisp en basit)	Daha karmaşık
Semantik	Basit (referential transparency)	Karmaşık (state bağımlı)
Değişkenler	Yok (saf FP)	Merkezi kavram
Tekrar	Recursion	Iteration ve recursion
Hız	Ortalama 2× yavaş	Von Neumann'a optimize
Concurrency	Doğal (state yok)	Synchronization zor
Test edilebilirlik	Kolay (fonksiyon bağımsız)	Bağlam bağımlı
Program boyutu	Çok daha kısa	Uzun

Readability karşılaştırması – İlk n pozitif tamsayının küplerinin toplamı:

Listing 25: C versiyonu – imperatif

```

1 int sum_cubes(int n) {
2     int sum = 0;
3     for (int i = 1; i <= n; i++)
4         sum += i * i * i;    // degisken yonetimi gerekli
5     return sum;
6 }

```

Listing 26: Haskell versiyonu – fonksiyonel

```

1 -- Sadece 3 adim: liste olustur, kupleri al, topla
2 sumCubes n = sum (map (^3) [1..n])

```

12 Bölüm Özeti: Tüm Kavramlar Bir Bakışta

Kavram	Tanım	Kritik Nokta	İlgili Diller
Referential transparency	Aynı girdi → her zaman aynı çıktı	<i>State bağımsızlığı</i>	Haskell, saf FP
Lambda expression	İsimsiz fonksiyon tanımı	<i>Fonksiyon = değer</i>	Tüm FP dilleri
CAR	Listenin ilk elemanı	<i>IBM 704 kökenli</i>	Lisp, Scheme
CDR	Listenin kuyruğu	<i>IBM 704 kökenli</i>	Lisp, Scheme
CONS	Eleman + liste → yeni liste	<i>CAR/CDR'nin tersi</i>	Lisp, Scheme
EVAL	Her ifadeyi değerlendiren fonksiyon	<i>Serendipitous yorumlayıcı</i>	Lisp, Scheme
QUOTE	Değerlendirmeyi engeller	<i>Kod=veri çakışması için</i>	Lisp, Scheme
DEFINE	İsim-değer veya isim-lambda bağlama	<i>Special form, 2 biçim</i>	Scheme
COND	Çok yollu seçim	<i>Else-if zinciri</i>	Scheme
Tail recursion	Son işlem özyineleme	<i>Stack büyümmez</i>	Scheme (zorunlu)
Type inference	Tip açıklamadan çıkarım	<i>Derleyici tipi bulur</i>	ML, Haskell, F#
Pattern matching	Parametre şekline göre fonksiyon seçimi	<i>Dallanma kolaylaşır</i>	ML, Haskell, F#
Currying	Çok parametreliliyi tek parametreliliye bölme	<i>Partial eval mümkün</i>	ML, Haskell, F#
Lazy evaluation	İfade yalnızca gerekince hesaplanır	<i>Sonsuz listeler mümkün</i>	Haskell
Dynamic scoping	Çağrıldığı ortamın değişkenleri	<i>Kazara seçildi</i>	Erken Lisp
Static scoping	Tanımlandığı ortamın değişkenleri	<i>Öngörülebilir</i>	Scheme, ML, Haskell, F#
First-class function	Fonksiyon = her yerde kullanılabilir değer	<i>Higher-order mümkün</i>	Scheme, ML, Haskell
Higher-order function	Fonksiyon alan veya döndüren fonksiyon	<i>map, filter, compose</i>	Tüm FP dilleri
REPL	Read-Eval-Print Loop	<i>İnteraktif yorumlayıcı</i>	Scheme, Python, Ruby
Pure FP	Değişken ve state yok	<i>Test çok kolay</i>	Haskell

13 Sınav İçin Kritik Noktalar

1. **CAR/CDR ismi:** IBM 704'teki donanım alanlarından gelir. Sınav sorusu: "Neden bu isimler?"
2. **QUOTE neden gerekli?** Scheme'de kod ve veri aynı formata sahip; QUOTE veriyi "dondurup" değerlendirilmesini engeller.
3. **EVAL'in serendipitous keşfi:** McCarthy teorik amaçla yazdı; Russell/Edwards yorumlayıcıya dönüştürdü.
4. **DEFINE special form:** Normal primitive gibi davranıyordu parametrelerini önce değerlendirirdi – ama tanımlanmamış isim hata verir.
5. **EQ? vs EQV? vs EQUAL?:** EQ? = pointer; EQV? = atom değer; EQUAL? = yapısal.
6. **Tail recursion:** Özyinelemeli çağrı son işlem → stack büyümeyiz → Scheme standardı bunu zorunlu kılar.
7. **Dynamic scoping yanlışlıkla seçildi:** Erken Lisp'te; Scheme 1975'te static scoping'e geçti.
8. **ML no overloading, no coercion:** square(2.75) int square için hata. F# da aynı – 2 ile 2.0 farkı kritik.
9. **Currying:** Çok parametreliliği tek parametreliliğe zincirine dönüştürme. Partial evaluation'ı mümkün kılar.
10. **Lazy evaluation = sonsuz listeler:** Yalnızca Haskell. Strict = Scheme, ML.
11. **Haskell saf FP, ML değil:** ML mutable array ve bazı yan etkiler içerir.
12. **F# rec zorunlu:** Özyinelemeli fonksiyon tanımında `rec` unutulursa derlenmez.
13. **LET = LAMBDA kısaltması:** $(\text{LET } ((a \ 7)) \text{ expr}) \equiv ((\text{LAMBDA } (a) \text{ expr}) \ 7)$
14. **Fonksiyonel diller neden yaygınlaşmadı?** İmparatif programcıların alışkanlıkları + early slow interpreters.
15. **Referential transparency:** Aynı argüman → her zaman aynı sonuç. Yan etki yok. Test kolay.

14 Review Questions — Soru Çözümleri

S1. Define functional form, simple list, bound variable, and referential transparency.

Cevap

Functional form (Higher-order function): Parametresi olarak fonksiyon alan veya sonuç olarak fonksiyon üreten fonksiyon. Örnekler: `map`, `filter`, `compose`.

Simple list: Elemanları yalnızca atomlardan oluşan, içinde alt liste bulunmayan liste. Örnek: `(A B C D)` basit listedir; `(A (B C) D)` basit liste değildir.

Bound variable: Lambda expression veya fonksiyon parametresi olarak belirli bir değere bağlanmış değişken. Değerlendirme sırasında sabit kalır, imperatif değişken gibi değiştirilemez.

Referential transparency: Bir ifadenin değeri yalnızca parametrelerine bağlıdır; programın durumuna (state) bağlı değildir. Aynı argümanlarla çağrılan fonksiyon her zaman aynı sonucu verir. Yan etki (side effect) yoktur.

S2. What does a lambda expression specify?

Cevap

Lambda expression, bir fonksiyonun **parametre listesini** ve o parametreler üzerindeki **eşleme ifadesini (mapping expression)** belirtir. İsim atanmadan fonksiyonu tanımlar – yani lambda expression, fonksiyonun kendisidir. Church’ün (1941) lambda calculus notasyonundan gelmektedir. Örnek: $\lambda(x) x \times x \times x$ – bu, x ’i alıp x^3 üreten isimsiz bir fonksiyondur.

S3. What data types were parts of the original Lisp?

Cevap

Orijinal Lisp’te yalnızca iki veri kategorisi vardı: **atoms** ve **lists**. Atoms, semboller (identifer) veya sayısal literallerdi. Listeler, parantezlerle sınırlandırılmış, birbirine bağlı elemanlardı. Orijinal Lisp **tipsiz (typeless)** bir dildi – tipler imperatif dillerdeki anlamda yoktu.

S4. In what common data structure are Lisp lists normally stored?

Cevap

Lisp listeleri bellekte **linked list (bağlantılı liste)** olarak saklanır. Her düğüm iki pointer içerir: biri veriye (ya da alt listeye), diğeri bir sonraki düğüme işaret eder. Listeye başından referans tutulur.

S5. Explain why QUOTE is needed for a parameter that is a data list.

Cevap

Scheme’de her ifade EVAL tarafından değerlendirilir. Bir liste yazıldığında yorumlayıcı onu *fonksiyon çağırısı* olarak yorumlar (ilk eleman fonksiyon adı, geri kalanı argüman). Ama bir listeyi **veri** olarak kullanmak istiyorsak – yani hesaplanmadan olduğu gibi kalmasını istiyorsak – QUOTE ile "dondururuz". Scheme’de kod ve veri aynı forma sahip olduğundan bu ayrım zorunludur. Kısaltması apostroftur: $'(A B C) = (\text{QUOTE } (A B C))$.

S6. What is a simple list?

Cevap

Basit liste (simple list), elemanları yalnızca atomlardan oluşan ve hiç alt liste içermeyen listedir. Örnek: $(A B C D)$. Buna karşın $(A (B C) D)$ basit liste değildir çünkü ikinci elemanı bir alt listedir.

S7. What does the abbreviation REPL stand for?

Cevap

REPL = Read-Evaluate-Print Loop. Scheme (ve Python, Ruby gibi) yorumlayıcısının çalışma modeli: kullanıcıdan bir ifade *okunur*, EVAL ile *değerlendirilir*, sonuç *yazdırılır*, ve döngü başa döner.

S8. What are the three parameters to IF?

Cevap

IF'in üç parametresi: **(1) predicate expression** (koşul ifadesi), **(2) then expression** (koşul doğruysa döndürülecek ifade), **(3) else expression** (koşul yanlışsa döndürülecek ifade). Sözdizimi: (IF predicate then-expr else-expr).

S9. What are the differences between =, EQ?, EQV?, and EQUAL??

Cevap

- **=** – Yalnızca sayısal eşitlik karşılaştırması. Sembolik atomlar için hata verir.
- **EQ?** – Pointer eşitliği. Sembolik atomlar için güvenilir; sayılar ve listeler için sonuç belirsiz (aynı pointer'a işaret edip etmediklerine bağlı).
- **EQV?** – Hem sembolik hem sayısal atomlar için değer karşılaştırması. EQ?'dan daha yavaş ama güvenilir. Listeler için hâlâ tutarsız.
- **EQUAL?** – Her tür için yapısal eşitlik karşılaştırması. Listeler dahil her yapıda çalışır. En yavaşıdır; yalnızca gerektiğinde kullanılmalıdır.

S10. What are the differences between the evaluation method used for DEFINE and that used for its primitive functions?

Cevap

DEFINE, EVAL'in **özel biçimde (special form olarak)** işlediği bir yapıdır. Normal bir primitive fonksiyon çağrısında EVAL önce tüm parametreleri değerlendirir, sonra fonksiyonu uygular. Eğer DEFINE de böyle çalışsaydı, (DEFINE x 10) çağrısında EVAL önce x'i değerlendirmeye çalışırdı – ama x henüz tanımlı olmadığından hata verirdi. DEFINE ise parametrelerini değerlendirmeden işler: sembolü tanım tablosuna ekler ve ifadeyi değerlendirerek ona bağlar.

S11. What are the two forms of DEFINE?

Cevap

Biçim 1 – İsme değer bağlama: (DEFINE symbol expression) – Bir ismi bir ifadenin değerine bağlar. Örnek: (DEFINE pi 3.14159).

Biçim 2 – Fonksiyon tanımlama: (DEFINE (fonksiyon-adı parametreler) (ifade)) – Lambda expression'ı bir isme bağlar. Bu biçim, LAMBDA kelimesini kaldırılmış kısaltılmış biçimdir.

S12. Describe the syntax and semantics of COND.

Cevap

COND, çok yollu seçim yapısıdır. Sözdizimi: (COND (pred1 expr1) (pred2 expr2) ... (ELSE exprN)). Anlambilimi: Predikatlar sırayla değerlendirilir; doğru (herhangi bir non-#F değer) olan ilk predikatın karşılığı olan ifade döndürülür. ELSE her zaman doğru kabul edilir. Eşleşen predicate yoksa ve ELSE de yoksa, sonuç belirsizdir.

S13. Why are CAR and CDR so named?

Cevap

Lisp'in ilk implementasyonunun yapıldığı **IBM 704** bilgisayarının bellek sözcükleri iki alan içeriyordu: *address part* ve *decrement part*. Makine bu alanları okuyan **CAR** (Contents of the Address part of a Register) ve **CDR** (Contents of the Decrement part of a Register)

komutlarını içeriyordu. Bir liste düğümü bu iki alana mükemmel biçimde sığıyordu. İsimler donanım komutlarından Lisp'e miras kaldı.

S14. If CONS is called with two atoms, what is returned?

Cevap

İki atom ile çağrılan CONS, **dotted pair (noktalı çift)** döndürür. Örnek: (CONS 'A 'B) sonucu (A . B) olarak gösterilir. Bu, bir liste düğümünün iki işaretçisinin de atom tuttuğunu belirtir. Normal listeler yerine noktalı çiftler genellikle kasıtlı kullanılmaz.

S15. Describe the syntax and semantics of LET in Scheme.

Cevap

Sözdizimi: (LET ((isim1 ifade1) (isim2 ifade2) ...) gövde-ifadesi)

Anlambilimi: LET, belirtilen isimleri belirtilen ifadelerin değerlerine bağlar ve gövde ifadesini bu bağlamda değerlendirir. Bağlamalar paralel oluşur (birbirinden bağımsız). Bağlanan isimler yeniden bağlanamaz. LET, LAMBDA'nın kısaltmasıdır: (LET ((a 7)) (* 5 a)) $\equiv$ ((LAMBDA (a) (* 5 a)) 7).

S16. What are the differences between CONS, LIST, and APPEND?

Cevap

- **CONS:** Tam olarak 2 parametre alır. Birinci parametre (atom ya da liste) yeni listenin başı (CAR'ı), ikinci parametre (liste) kuyruğu (CDR'si) olur. Yeni liste tek adımda oluşturulur.
 - **LIST:** Değişken sayıda parametre alır. Tüm parametreleri elemanları olan yeni bir liste oluşturur. İçeride birden fazla CONS çağrısına karşılık gelir.
 - **APPEND:** İki liste parametre alır ve bunları *birleştirerek* yeni bir liste döndürür. Birinci listenin tüm elemanları ardından ikincinin elemanları gelir. Özyineleme kullanır.
- Örnek farkı: (CONS '(A B) '(C D)) $\rightarrow$ ((A B) C D); (APPEND '(A B) '(C D)) $\rightarrow$ (A B C D).

S17. Describe the syntax and semantics of mapcar in Scheme (map).

Cevap

Sözdizimi: (map fonksiyon liste)

Anlambilimi: map, verilen fonksiyonu listenin her elemanına sırayla uygular ve sonuçlardan oluşan yeni bir liste döndürür. Orijinal liste değiştirilmez. Örnek: (map (LAMBDA (x) (* x x)) '(1 2 3 4)) $\rightarrow$ (1 4 9 16).

Map, matematiksel apply-to-all (herkese uygula) işleminin Scheme karşılığıdır.

S18. What is tail recursion? Why is it important?

Cevap

Tail recursion (kuyruk özyinelemesi): Özyinelemeli çağrının fonksiyondaki *son işlem* olması durumudur. Son işlem olduğundan, çağrıldıktan sonra mevcut activation record'a geri dönmeye gerek yoktur.

Önemi: Tail recursive fonksiyonlar, stack üzerinde alan büyütmeden iterasyona dönüştürülebilir. Bu, derin özyinelemelerde (örn. büyük listeler) stack overflow riskini ortadan kaldırır ve bellek kullanımını dramatik biçimde azaltır. Scheme standardı, sistemin

tüm tail recursive fonksiyonları iterasyona çevirmesini **zorunlu** kılar.

S19. Why were imperative features added to most dialects of Lisp?

Cevap

Saf fonksiyonel programlama, bazı problem türleri için verimsizdi: özellikle I/O işlemleri, sayısal hesaplamalar ve büyük veri yapılarının yerinde güncellenmesi gibi durumlarda saf özyinelemeli yaklaşım çok yavaş kalıyordu. İmperatif özellikler (atama deyimleri, döngüler, mutable değişkenler) bu tür durumlarda **çalışma zamanı verimliliğini** ve **kullanım kolaylığını** artırmak amacıyla eklendi.

S20. In what ways are Common Lisp and Scheme opposites?

Cevap

- **Boyut:** Scheme küçük ve sade; Common Lisp büyük ve karmaşık (C++ benzeri).
- **Scoping:** Scheme yalnızca static scoping; Common Lisp hem static hem dynamic scoping sunar.
- **Amaç:** Scheme eğitim odaklı; Common Lisp ticari uygulama odaklıdır.
- **Tasarım felsefesi:** Scheme minimalist; Common Lisp eski Lisp lehçelerinin birleşimi.

S21. What scoping rule is used in Scheme? In Common Lisp? In ML? In Haskell? In F#?

Cevap

- **Scheme:** Static (lexical) scoping.
- **Common Lisp:** Varsayılan static scoping; `special` ile dynamic scoping seçilebilir.
- **ML:** Static scoping.
- **Haskell:** Static scoping.
- **F#:** Static scoping.

Kısaca: tüm çağdaş fonksiyonel diller static scoping kullanır.

S22. What happens during the reader phase of a Common Lisp language processor?

Cevap

Common Lisp'te **reader (okuyucu) aşaması**: Programın metin formu, code representation (kod gösterimi) adı verilen dahili bir forma dönüştürülür. Bu aşamada **reader macro** adı verilen özel makrolar devreye girer. Bir reader macro, belirli bir karakteri Lisp koduna genişletir. Örneğin apostrof (') bir reader macro'dur ve (QUOTE ...) çağrısına genişler. Kullanıcılar kendi reader macro'larını tanımlayabilir.

S23. What are two ways that ML is fundamentally different from Scheme?

Cevap

1. **Güçlü tiplendirme:** ML güçlü tipli bir dildir; her değişken ve ifadenin tipi statik olarak belirlenebilir. Scheme ise esasen tipsizdir.
2. **Sözdizimi:** ML, infix aritmetik gibi imperatif dillere yakın sözdizimini kullanır. Scheme ise Lisp'ten gelen önek (prefix) notasyonunu kullanır.

S24. What is stored in an ML evaluation environment?

Cevap

ML'deki **evaluation environment (değerlendirme ortamı)**, programdaki açık veya örtük biçimde tanımlanmış tüm tanımlayıcıların (identifier) *isimlerini* ve *tiplerini* saklayan bir tablodur. Çalışma zamanındaki sembol tablosu gibi düşünülebilir. Bir tanımlayıcı tanımlandığında bu tabloya eklenir.

S25. What is the difference between an ML val statement and an assignment statement in C?

Cevap

ML val: Bir ismi bir değere bağlar; bu bağlama *sonradan değiştirilemez*. Yeni bir **val** ile aynı isim yeniden bağlanırsa bu yeni bir giriş oluşturur, eskisi geçersiz hale gelir ama değiştirilmez. Yan etkisi yoktur.

C atama (=): Bir bellek hücresinin değerini değiştirir. Yan etkisi vardır; aynı değişken programın farklı noktalarında farklı değerlere sahip olabilir.

S26. What is type inferencing, as used in ML?

Cevap

Type inferencing (tip çıkarımı): Derleyicinin, değişken ve parametre tiplerini programcı belirtmeden, ifadedeki operatörler ve literaller aracılığıyla otomatik olarak belirleme yeteneğidir. Örnek: `fun times10(x) = 10 * x - 10` integer olduğundan, `x` ve dönüş tipi de `int` olarak çıkarılır. ML, OCaml, Haskell ve F# bu özelliği destekler.

S27. What is the use of the `fn` reserved word in ML?

Cevap

fn, ML'de **isimsiz (anonymous) lambda expression** tanımlamak için kullanılır. `fun` ile adlandırılmış fonksiyon tanımlanırken, `fn` ile parametre ve gövdesi belirtilen isimsiz bir fonksiyon oluşturulur. Örnek: `fn(x) => x < 100 - x`'in 100'den küçük olup olmadığını test eden isimsiz fonksiyon.

S28. Can ML functions that deal with scalar numerics be generic?

Cevap

Hayır, ML'de skaler sayısal işlemler generic (polimorfik) olamaz. Örneğin `fun square(x) = x * x` tanımlandığında ML, `*` operatöründen `int` tipini çıkarsar. Bu fonksiyon artık yalnızca `int` için çalışır; `real` için ayrı tanım gerekir. Ayrıca ML fonksiyon aşırı yüklemeye (overloading) izin vermez, bu nedenle her iki versiyon bir arada bulunamaz.

S29. What is a curried function?

Cevap

Curried function (köri fonksiyon): Birden fazla parametrelili bir fonksiyonun, tek parametrelili ve geriye fonksiyon döndüren fonksiyonlar zinciriyle ifade edilmiş biçimidir. Haskell Curry'nin adından gelir. Örnek: `fun add a b = a + b` görünürde iki parametrelili ama aslında `a` alıp `b → a+b` fonksiyonu döndüren tek parametrelili bir fonksiyon. Çağrı: `add 3 5 → 8`.

S30. What does partial evaluation mean?

Cevap

Partial evaluation (kısmi değerlendirme): Curried bir fonksiyona tüm parametrelerinden daha az argüman verilerek yeni, özelleşmiş bir fonksiyon elde etme işlemidir. Örnek: `fun add5 x = add 5 x` – `add` fonksiyonuna ilk parametre olarak 5 verilerek, "herhangi bir sayıya 5 ekle" işlevini yapan yeni `add5` fonksiyonu üretilir.

S31. Describe the actions of the ML filter function.

Cevap

`filter`, iki parametrelili higher-order bir fonksiyondur: (1) **predicate fonksiyon** ve (2) **liste**. Listedeki her eleman predikat fonksiyonuna uygulanır; predikat `true` döndüren elemanlardan oluşan yeni bir liste üretilir. Orijinal liste değişmez. Örnek: `filter(fn(x) => x < 100)` `[25, 1, 50, 711, 100, 150, 27] → [25, 1, 50, 27]`.

S32. What operator does ML use for Scheme's CAR?

Cevap

ML, Scheme'nin `CAR`'ına karşılık `hd` (head) fonksiyonunu kullanır. Benzer şekilde `CDR`'a karşılık `tl` (tail), ve `CONS`'a karşılık `::` operatörü kullanılır.

S33. What operator does ML use for functional composition?

Cevap

ML'de fonksiyon bileşimi için `o` operatörü (küçük harf "o") kullanılır. Sözdizimi: `val h = g o f` – bu, `h(x) = g(f(x))` anlamına gelir. Önce `f` uygulanır, çıktısı `g`'ye verilir.

S34. What are the three characteristics of Haskell that make it different from ML?

Cevap

1. **Nonstrict (lazy) semantics:** Haskell lazy evaluation kullanır; ifadeler yalnızca gerektiğinde değerlendirilir. ML strict'tir.
2. **Function overloading:** Haskell, type class mekanizmasıyla fonksiyon aşırı yüklemeyi destekler. ML bunu desteklemez.
3. **Pure functional:** Haskell saf fonksiyonel bir dildir – yan etki içeren ifadeler ve değişken yoktur. ML ise mutable diziler ve bazı yan etkiler içerir.

S35. What does lazy evaluation mean?

Cevap

Lazy evaluation (tembel değerlendirme): Bir ifade yalnızca değerine gerçekten ihtiyaç duyulduğunda değerlendirilir. Eğer bir parametrenin değerine hiç ihtiyaç duyulmazsa, o parametre hiç hesaplanmaz. Parametrenin yalnızca bir parçasına ihtiyaç duyulursa, yalnızca o parça hesaplanır. Lazy evaluation, sonsuz veri yapılarının (infinite lists) kullanılabilmesini sağlar.

S36. What is a strict programming language?

Cevap

Strict (katı) dil: Fonksiyon çağrılmadan önce tüm gerçek parametrelerin tam olarak değerlendirilmesini zorunlu kılan dil. Bu, parametrelerin değerlendirilme sırasının fonksiyonun

değerini etkilememesini garanti eder. Scheme ve ML strict'tir.

S37. What programming paradigms are supported by F#?

Cevap

F# üç temel programlama paradigmasını destekler: **(1) Fonksiyonel programlama** (çekirdeği OCaml'dan gelen), **(2) İmparatif programlama** (mutable değişkenler, döngüler), **(3) Nesne yönelimli programlama** (.NET class sistemi üzerinden).

S38. With what other programming languages can F# interoperate?

Cevap

F#, tüm **.NET dilleri** ile tam birlikte çalışabilir. Bu, özellikle C# ve VB.NET'i kapsar. F# sınıfları diğer .NET dillerinden kullanılabilir ve subclass yapılabilir; tersine de mümkündür. F# programları tüm .NET Framework API'lerine erişebilir.

S39. What does F#'s let do?

Cevap

F#'taki **let**, bir ismi bir değere bağlar – tıpkı ML'deki gibi. Hem değer hem de fonksiyon tanımlamak için kullanılır: **let pi = 3.14159** (sabit değer), **let rec factorial x = ...** (fonksiyon). Bağlama mutable değildir; ancak aynı isim daha sonra outscoping ile yeniden bağlanabilir.

S40. How is the scope of a F# let construct terminated?

Cevap

F#'ta bir **let** bağlamasının kapsamı, aynı isim başka bir **let** ile **yeniden bağlandığında** (**outscoping**) sona erer. İkinci **let** ilkinin kapsamını kapatır ve yeni bir bağlama oluşturur. Bu, ML'deki **val**'den farklıdır: ML'de bir **val** bağlaması blok sona erene kadar geçerliydi.

S41. What is the underlying difference between a sequence and a list in F#?

Cevap

List: Tüm elemanlar bellekte tam olarak saklanır (eager/greedy). Sonlu olmalıdır.
Sequence (seq<T>): Lazy biçimde üretilir – elemanlar yalnızca ihtiyaç duyulduğunda hesaplanır. Bu nedenle *sonsuz* sequence tanımlanabilir (**seq {0..}**). Sequence, .NET'in **IEnumerable** arayüzüne karşılık gelir.

S42. What is the difference between the let of ML and that of F#, in terms of extent?

Cevap

ML val: Bağlamanın kapsamı, bulunduğu bloğun sonuna kadar devam eder ve dış blokta geçerli olmaz.
F# let: Bağlamanın kapsamı, aynı isim yeniden bağlanana kadar (outscoping) devam eder. Bu nedenle F#'ta bir isim, aynı fonksiyon içinde birden fazla kez **let** ile yeniden bağlanabilir.

S43. What is the syntax of a lambda expression in F#?

Cevap

F#'ta lambda expression **fun** anahtar sözcüğüyle tanımlanır:

fun parametre(ler) -> ifade

Örnek: (fun a b -> a / b) – a'yı b'ye bölen isimsiz fonksiyon. Birden fazla parametre varsa aralarına boşluk konur. -> operatörü ile gövde ayrılır.

S44. Does F# coerce numeric values in expressions?

Cevap

Hayır, F# sayısal tür dönüşümü yapmaz. Örneğin `let x = 2 * 3.14` hata verir; `2.0 * 3.14` yazılmalıdır. Bu tasarım kararı ML'in mirasından gelir ve tür güvenliğini (type safety) güçlendirir; örtük dönüşümlerden kaynaklanabilecek beklenmedik davranışları önler.

S45. What support does Python provide for functional programming?

Cevap

Python şu fonksiyonel özellikleri destekler:

- **Lambda expressions:** `lambda params: expr` sözdizimi.
- **Higher-order functions:** `map()`, `filter()` built-in.
- **List comprehensions:** `[x**2 for x in range(10)]`.
- **Partial application:** `functools.partial()` ile.
- **Closures:** Lambda ve iç fonksiyonlar dış değişkenleri kapatabilir.

S46. What function in Ruby is used to create a curried function?

Cevap

Ruby'de **curry** metodu, bir lambda veya proc nesnesini curry edilmiş biçimine dönüştürür. Örnek: `times = lambda {|a, b| a * b} → times5 = times.curry.(5)` – bu, "herhangi bir sayıyı 5 ile çarp" işlevini yapan yeni bir fonksiyon üretir.

S47. Is the use of functional programming expanding or shrinking?

Cevap

Genişliyor. Özellikle son on yılda, ML, Haskell, OCaml ve F# gibi tipli fonksiyonel dillerin olgunlaşmasıyla birlikte kullanım alanları genişledi. Ayrıca Python, Java, C# gibi imperatif dillerin lambda, map, filter gibi fonksiyonel özellikler eklemesi, fonksiyonel paradigmanın ana akıma girdiğini göstermektedir.

S48. What is one characteristic of functional programming languages that makes their semantics simpler?

Cevap

Referential transparency: Saf fonksiyonel dillerde bir ifadenin anlamı yalnızca kendi içindeki yapıya bağlıdır; programın yürütme geçmişine, değişken değerlerine ya da dış bağlama bağlı değildir. İmperatif bir dilde basit bir atama deyimini anlamlandırmak için bile değişken kapsamını, scoping kurallarını ve programın önceki durumunu incelemek gerekebilir. Fonksiyonel dillerde bu tür bağlamsal analiz gerekmez.

S49. What is the flaw in using lines of code to compare productivity?

Cevap

Kod satır sayısı, üretkenliğin (productivity) güvenilir bir ölçütü değildir çünkü tüm satırlar eşit karmaşıklığa ve yazmaya harcanan zamana sahip değildir. Fonksiyonel programlarda kısa ama derin satırlar, imperatif programlarda ise çok sayıda basit değişken başlatma/güncelleme satırları bulunabilir. Dolayısıyla "10 kat daha kısa = 10 kat daha üretken" çıkarımı yanlış olabilir.

S50. Why can concurrency be easier with functional languages than imperative languages?

Cevap

Saf fonksiyonel dillerde değişken ve paylaşımlı mutable state yoktur. Bu nedenle iki fonksiyon birbirinin sonucunu etkileyemez – bağımsızdırlar. Bu bağımsızlık, hangi fonksiyonların *eş zamanlı* çalıştırılabileceğinin tespitini kolaylaştırır; ayrıca senkronizasyon mekanizmalarına (kilitleme, mutex gibi) ihtiyaç kalmaz. İmperatif dillerde ise paylaşımlı değişkenler üzerindeki yarış koşulları (race conditions) concurrency'yi karmaşık ve hataya açık hale getirir.

15 Problem Set — Soru Çözümleri

P1. Read Backus's FP paper (1978) and compare features of Scheme with FP.

Cevap

Backus'un 1978 tarihli makalesi "*Can Programming Be Liberated from the von Neumann Style?*" şu temel argümanı öne sürer: İmperatif diller von Neumann darboğazına sıkışmış durumdadır (tek tek bellek hücrelerini güncellemek zorundayız). FP dili bunu aşmak için tasarlanmıştır.

Scheme ile karşılaştırma:

- *Benzerlikler:* Her ikisi de higher-order fonksiyonlar, function composition ve apply-to-all (map) içerir. Her ikisi de özyinelemeye dayanır.
- *Farklar:* Backus'un FP'si daha kısıtlayıcıdır – değişken yoktur, hiçbir biçimde. Scheme ise **set!** ile mutable değişken sunar. FP'de lambda expression yoktur; fonksiyonlar yalnızca kombinatörler aracılığıyla oluşturulur. Scheme'de sözdizimsel esneklik çok daha fazladır. FP hiçbir zaman yaygın kullanıma ulaşamadı; Scheme ise eğitim alanında standard haline geldi.

P2. Find definitions of EVAL and APPLY and explain their actions.

Cevap

EVAL: Bir Scheme ifadesini argüman olarak alır ve o ifadeyi mevcut ortamda değerlendirir. Literal sayılar kendini döndürür; semboller değerleri döndürür; listeler fonksiyon çağırısı olarak işlenir. Tüm Scheme yorumlayıcısının çekirdeğidir. Scheme 4'ten itibaren ikinci parametre olarak ortam (environment) da alır.

APPLY: Bir fonksiyon ve bir argüman listesi alır; fonksiyonu liste elemanlarına uygular. Sözdizimi: (APPLY + '(1 2 3)) → 6. APPLY, fonksiyon uygulamasını liste üzerinden gerçekleştirmenin yoludur. EVAL ile birlikte Scheme'nin meta-programlama çekirdeğini oluştururlar.

P3. Compare difficulty of writing Lisp programs on your system with INTERLISP.

Cevap

INTERLISP (Teitelman & Masinter, 1981), entegre bir programlama ortamı sunan gelişmiş bir Lisp sistemidir. Standart Lisp/Scheme sistemlerinden farkları: (1) **Structure editor** – kodu metin olarak değil, liste yapısı olarak düzenler; sözdizimi hatası imkansızdır. (2) **Integrated debugger** – çalışma zamanında fonksiyon inceleme ve değiştirme. (3) **Undo mekanizması** – her işlem geri alınabilir. (4) **Otomatik doğrulama** – baskı hataları otomatik düzeltilebilir.

Kıyasla modern DrRacket ya da SLIME/SBCL kombinasyonu benzer kolaylıklar sağlar; ancak INTERLISP'in 1981 standartlarına göre bu özellikler son derece ileri sayılırdı.

P4. What arguments support the inclusion of the PROG feature in Lisp?

Cevap

PROG, Lisp'e eklenen imperatif bir yapıdır: sıralı ifadeler çalıştırmayı, etiket (label) ve GO ile koşulsuz dallanmayı sağlar. Dahil edilmesini destekleyen argümanlar: (1) Bazı algoritmalar (özellikle sayısal ve I/O yoğun) saf özyineleme ile ifade edildiğinde hem yavaş hem de okunması güç olur; PROG bu durumlarda pratik alternatif sunar. (2) Lisp'in AI uygulamalarında geniş benimsenmesi için pratik programlama kolaylığı gerekmektedir. (3) Imperatif programcıların Lisp'e geçişini kolaylaştırır.

P5. Find commercial systems using typed functional languages in database, financial, statistical, and bioinformatics areas.

Cevap

- **Veritabanı:** Erlang (Ericsson) telecom veritabanları için; CouchDB Erlang ile yazılmıştır.
- **Finansal modelleme:** Jane Street Capital, büyük ölçüde **OCaml** kullanan önde gelen algoritmik trading firmasıdır. F# Microsoft'ta finansal analiz araçlarında kullanılmaktadır.
- **İstatistiksel analiz:** Haskell, istatistik kütüphaneleri (statistics paketi) ile akademik çevrelerde kullanılır.
- **Biyoinformatik:** OCaml, Gen araştırma araçları için (örn. BLAST varyantları) kullanılmıştır. Haskell'de biyoinformatik kütüphaneler (BioHaskell) mevcuttur.

P6. What primitives would a functional language use if it used strings instead of lists?

Cevap

Listeler yerine tek karakterli sembollerin string'leri kullanılsaydı, CAR/CDR/CONS'un karşılığı şöyle olurdu:

- **FIRST-CHAR** (CAR karşılığı): String'in ilk karakterini döndürür.
- **REST-STR** (CDR karşılığı): İlk karakter dışında kalan alt string'i döndürür.
- **PREPEND** (CONS karşılığı): Bir karakteri string'in başına ekler.
- **EMPTY?** (NULL? karşılığı): Boş string (") mı kontrolü.
- **CONCAT** (APPEND karşılığı): İki string'i birleştirir.

P7. Make a list of features of F# that are not in ML.

Cevap

- .NET framework entegrasyonu ve tüm .NET API'lerine erişim.
- Nesne yönelimli programlama (sınıflar, kalıtım) tam desteği.
- Pipeline operatörü |> – zincir fonksiyon çağrısı.
- Fonksiyon kompozisyonu için » operatörü.
- Sequence expressions (lazy generator) sözdizimi: `seq {for i in 1..10 -> ...}`.

- Diğer .NET dilleriyle tam birlikte çalışabilirlik.
- Visual Studio tam IDE desteği.
- Discriminated unions'ın genişletilmiş biçimi.
- Outscoping ile aynı ismin yeniden bağlanabilmesi.

P8. If Scheme were a pure functional language, could it include DISPLAY? Why or why not?

Cevap

Hayır, içeremez. DISPLAY, ekrana çıktı üretir – bu bir **yan etki (side effect)** dir. Saf fonksiyonel programlama tanım gereği yan etki içermez; fonksiyonlar yalnızca girdilere bağlı olarak değer üretir, dış dünyayı (ekran, dosya, ağ) etkilemez. Aynı şekilde giriş (input) operasyonları da program durumunu değiştirdiğinden saf FP ile uyumsuz. Haskell bu sorunu **monad** sistemiyle çözer: I/O operasyonları tipler aracılığıyla sistemin geri kalanından izole edilir.

P9. What does the following Scheme function y do?

Listing 27: Fonksiyon y – ne yapar?

```
1 (define (y s lis)
2   (cond
3     ((null? lis) '())
4     ((equal? s (car lis)) lis)
5     (else (y s (cdr lis)))))
```

Cevap

Fonksiyon y, s sembolünü lis listesinde arar. Eşleşme bulunduğunda, **o elemandan itibaren listenin geri kalanını** döndürür (sembol dahil). Sembol listede yoksa boş liste () döndürür. Bu işlem *member* fonksiyonuna benzer; ancak member #T/#F döndürürken bu fonksiyon listenin bir alt dizisini döndürür.

Örnekler:

- (y 'B '(A B C D)) → (B C D)
- (y 'E '(A B C D)) → ()

P10. What does the following Scheme function x do?

Listing 28: Fonksiyon x – ne yapar?

```
1 (define (x lis)
2   (cond
3     ((null? lis) 0)
4     ((not (list? (car lis)))
5      (cond
6        ((eq? (car lis) #f) (x (cdr lis)))
7        (else (+ 1 (x (cdr lis)))))
8     (else (+ (x (car lis)) (x (cdr lis))))))
```

Cevap

Fonksiyon x, listede **#F olmayan** (yani truthy) **atom sayısını** hesaplar. İç içe listeler de dahil tüm yapıyı recursive olarak tarar. Algoritma: liste boşsa 0 döndür; mevcut eleman atom ise ve #F değilse 1 ekleyip devam et; #F ise atla; eleman liste ise o listeye de aynı işlemi uygula.

Örnekler:

- (x '(A B C)) → 3

- $(x \text{ '}(A \text{ #F } B \text{ #F } C)) \rightarrow 3$
- $(x \text{ '}(A \text{ (B } C) \text{ D})) \rightarrow 4$

16 Programming Exercises — Çözümler

Listing 29: PE1 – Kure hacmi

```
1 ;; V = (4/3) * pi * r^3
2 (DEFINE (sphere-volume r)
3   (* (/ 4.0 3.0) 3.14159 r r r))
4
5 (sphere-volume 5) ; => 523.598...
```

Listing 30: PE2 – İkinci derece denklem kokleri (IF ile)

```
1 (DEFINE (quadratic-if a b c)
2   (LET ((discriminant (- (* b b) (* 4 a c))))
3     (IF (< discriminant 0)
4       (display "Kokler karmasik")
5       (LIST
6         (/ (+ (- 0 b) (sqrt discriminant)) (* 2 a))
7         (/ (- (- 0 b) (sqrt discriminant)) (* 2 a))))))
8
9 (quadratic-if 1 -5 6) ; => (3.0 2.0)
10 (quadratic-if 1 0 1) ; => Kokler karmasik
```

Listing 31: PE3 – İkinci derece denklem kokleri (COND ile)

```
1 (DEFINE (quadratic-cond a b c)
2   (LET ((disc (- (* b b) (* 4 a c))))
3     (COND
4       ((< disc 0) (display "Kokler karmasik"))
5       ((= disc 0) (LIST (/ (- 0 b) (* 2 a))))
6       (ELSE
7        (LIST
8          (/ (+ (- 0 b) (sqrt disc)) (* 2 a))
9          (/ (- (- 0 b) (sqrt disc)) (* 2 a)))))))
```

Listing 32: PE4 – A uzeri B kuvveti

```
1 (DEFINE (power a b)
2   (COND
3     ((= b 0) 1)
4     ((= b 1) a)
5     (ELSE (* a (power a (- b 1)))))
6
7 (power 2 10) ; => 1024
8 (power 3 4) ; => 81
```

Listing 33: PE5 – Listede sifir sayisi

```
1 (DEFINE (count-zeros lst)
2   (COND
3     ((NULL? lst) 0)
4     ((= (CAR lst) 0) (+ 1 (count-zeros (CDR lst))))
5     (ELSE (count-zeros (CDR lst))))
6
7 (count-zeros '(0 1 0 2 0 3)) ; => 3
```

Listing 34: PE6 – Listede en büyük ve en küçük

```

1 (DEFINE (max-min lst)
2   (DEFINE (helper lst cur-max cur-min)
3     (COND
4       ((NULL? lst) (LIST cur-max cur-min))
5       (ELSE (helper
6               (CDR lst)
7               (IF (> (CAR lst) cur-max) (CAR lst) cur-max)
8               (IF (< (CAR lst) cur-min) (CAR lst) cur-min))))))
9   (helper (CDR lst) (CAR lst) (CAR lst)))
10
11 (max-min '(3 1 4 1 5 9 2 6)) ; => (9 1)

```

Listing 35: PE7 – Ust duzey atomlari sil

```

1 ;; Listeden tum ust duzey atom orneklerini sil
2 (DEFINE (delete-top atm lst)
3   (COND
4     ((NULL? lst) '())
5     ((EQ? atm (CAR lst))
6      (delete-top atm (CDR lst))) ; bu elemani atla
7     (ELSE
8      (CONS (CAR lst) (delete-top atm (CDR lst))))))
9
10 (delete-top 'A '(A B A C A)) ; => (B C)

```

Listing 36: PE8 – Son elemani sil

```

1 (DEFINE (delete-last lst)
2   (COND
3     ((NULL? lst) '())
4     ((NULL? (CDR lst)) '()) ; tek eleman -> bos liste
5     (ELSE (CONS (CAR lst)
6                  (delete-last (CDR lst))))))
7
8 (delete-last '(A B C D)) ; => (A B C)

```

Listing 37: PE9 – Atom veya liste silinmesi (derinlige iner)

```

1 (DEFINE (delete-deep target lst)
2   (COND
3     ((NULL? lst) '())
4     ((EQUAL? target (CAR lst)) ; basta eslesme
5      (delete-deep target (CDR lst)))
6     ((LIST? (CAR lst)) ; alt liste ise ic ine in
7      (CONS (delete-deep target (CAR lst))
8            (delete-deep target (CDR lst))))
9     (ELSE
10      (CONS (CAR lst) (delete-deep target (CDR lst))))))
11
12 (delete-deep 'A '(A B (A C) D A)) ; => (B (C) D)

```

Listing 38: PE10 – Tum atom orneklerini degistir (derin)

```

1 (DEFINE (replace-deep old new lst)
2   (COND
3     ((NULL? lst) '())
4     ((EQUAL? old (CAR lst))
5      (CONS new (replace-deep old new (CDR lst))))
6     ((LIST? (CAR lst))

```

```

7      (CONS (replace-deep old new (CAR lst))
8            (replace-deep old new (CDR lst))))
9      (ELSE
10       (CONS (CAR lst) (replace-deep old new (CDR lst)))))
11
12 (replace-deep 'A 'Z '(A B (A C) D A)) ; => (Z B (Z C) D Z)

```

Listing 39: PE11 – Basit listeyi tersine çevir

```

1 (DEFINE (reverse-list lst)
2   (DEFINE (rev-helper lst acc)
3     (IF (NULL? lst)
4         acc
5         (rev-helper (CDR lst) (CONS (CAR lst) acc))))
6   (rev-helper lst '()))
7
8 (reverse-list '(A B C D)) ; => (D C B A)
9 ;; Tail-recursive versiyon (akumulator ile)

```

Listing 40: PE12 – Yapısal eşitlik testi

```

1 ;; İki listenin yapısal olarak eşit olup olmadığını test eder
2 ;; (atomlar farklı olabilir, yapı aynı olmalı)
3 (DEFINE (struct-equal? l1 l2)
4   (COND
5     ((AND (NULL? l1) (NULL? l2)) #T)
6     ((OR (NULL? l1) (NULL? l2)) #F)
7     ((AND (LIST? (CAR l1)) (LIST? (CAR l2)))
8      (AND (struct-equal? (CAR l1) (CAR l2))
9            (struct-equal? (CDR l1) (CDR l2))))
10    ((OR (LIST? (CAR l1)) (LIST? (CAR l2))) #F)
11    (ELSE (struct-equal? (CDR l1) (CDR l2))))
12
13 (struct-equal? '(A (B C) D) '(X (Y Z) W)) ; => #T
14 (struct-equal? '(A B) '(X (Y))) ; => #F

```

Listing 41: PE13 – İki kümenin birleşimi

```

1 (DEFINE (union set1 set2)
2   (COND
3     ((NULL? set1) set2)
4     ((member (CAR set1) set2) ; zaten set2'de var
5      (union (CDR set1) set2))
6     (ELSE
7      (CONS (CAR set1) (union (CDR set1) set2)))))
8
9 (union '(A B C) '(B C D E)) ; => (A B C D E)

```

Listing 42: PE14 – Tüm atomları sil (derin, yerine koyma yok)

```

1 (DEFINE (delete-atom-deep target lst)
2   (COND
3     ((NULL? lst) '())
4     ((AND (NOT (LIST? (CAR lst))) (EQ? (CAR lst) target))
5      (delete-atom-deep target (CDR lst)))
6     ((LIST? (CAR lst))
7      (LET ((cleaned (delete-atom-deep target (CAR lst))))
8        (CONS cleaned (delete-atom-deep target (CDR lst)))))
9     (ELSE
10      (CONS (CAR lst) (delete-atom-deep target (CDR lst)))))
11

```

```
12 (delete-atom-deep 'A '(A B (A C A) D A)) ; => (B (C) D)
```

Listing 43: PE15 – İkinci üst düzey eleman kaldır

```
1 (DEFINE (remove-second lst)
2   (COND
3     ((OR (NULL? lst) (NULL? (CDR lst))) '()) ; 2 eleman yok
4     (ELSE (CONS (CAR lst) (CDDR lst)))))
5
6 (remove-second '(A B C D)) ; => (A C D)
7 (remove-second '(A)) ; => ()
```

Listing 44: PE16 – Sayıları artan sırada sirala (insertion sort)

```
1 (DEFINE (insert n sorted-lst)
2   (COND
3     ((NULL? sorted-lst) (LIST n))
4     ((<= n (CAR sorted-lst)) (CONS n sorted-lst))
5     (ELSE (CONS (CAR sorted-lst) (insert n (CDR sorted-lst))))))
6
7 (DEFINE (isort lst)
8   (COND
9     ((NULL? lst) '())
10    (ELSE (insert (CAR lst) (isort (CDR lst)))))
11
12 (isort '(3 1 4 1 5 9 2 6)) ; => (1 1 2 3 4 5 6 9)
```

Listing 45: PE17 – En büyük ve en küçük (PE6 tekrarı, farklı versiyon)

```
1 (DEFINE (largest lst)
2   (IF (NULL? (CDR lst))
3       (CAR lst)
4       (MAX (CAR lst) (largest (CDR lst)))))
5
6 (DEFINE (smallest lst)
7   (IF (NULL? (CDR lst))
8       (CAR lst)
9       (MIN (CAR lst) (smallest (CDR lst)))))
10
11 (largest '(3 1 4 1 5 9)) ; => 9
12 (smallest '(3 1 4 1 5 9)) ; => 1
```

Listing 46: PE18 – Tüm permutasyonlar

```
1 (DEFINE (remove-elem x lst)
2   (COND
3     ((NULL? lst) '())
4     ((EQ? x (CAR lst)) (CDR lst))
5     (ELSE (CONS (CAR lst) (remove-elem x (CDR lst)))))
6
7 (DEFINE (permutations lst)
8   (COND
9     ((NULL? lst) '(()))
10    (ELSE
11      (apply append
12        (map (LAMBDA (x)
13              (map (LAMBDA (p) (CONS x p))
14                  (permutations (remove-elem x lst))))
15              lst))))))
16
17 (permutations '(A B C))
```

```
18 ;; => ((A B C) (A C B) (B A C) (B C A) (C A B) (C B A))
```

Listing 47: PE19 – Quicksort

```
1 (DEFINE (quicksort lst)
2   (COND
3     ((OR (NULL? lst) (NULL? (CDR lst))) lst)
4     (ELSE
5       (LET ((pivot (CAR lst))
6             (rest (CDR lst)))
7         (append
8           (quicksort (filter (LAMBDA (x) (< x pivot)) rest))
9           (LIST pivot)
10          (quicksort (filter (LAMBDA (x) (>= x pivot)) rest)))))))
11
12 (quicksort '(3 1 4 1 5 9 2 6 5)) ; => (1 1 2 3 4 5 5 6 9)
```

Listing 48: PE20 – doit fonksiyonunu tail-recursive yap

```
1 ;; Original (tail-recursive degil):
2 (DEFINE (doit n)
3   (IF (= n 0) 0 (+ n (doit (- n 1)))))
4
5 ;; Tail-recursive versiyon (accumulator ile):
6 (DEFINE (doit-helper n acc)
7   (IF (= n 0)
8       acc
9       (doit-helper (- n 1) (+ n acc))))
10
11 (DEFINE (doit n) (doit-helper n 0))
12
13 (doit 10) ; => 55 (0+1+2+...+10)
14 ;; Son islem ozyineleme -> stack buyumez!
```

Listing 49: PE21 – Quicksort F#'ta

```
1 // F# ile quicksort (list comprehension benzeri)
2 let rec quicksort lst =
3   match lst with
4   | [] | [_] -> lst
5   | h :: t ->
6     let smaller = List.filter (fun x -> x < h) t
7     let bigger  = List.filter (fun x -> x >= h) t
8     quicksort smaller @ [h] @ quicksort bigger;;
9
10 quicksort [3; 1; 4; 1; 5; 9; 2; 6]
11 // => [1; 1; 2; 3; 4; 5; 6; 9]
```

Listing 50: PE22 – reverse ML'de (tail-recursive)

```
1 (* ML ile ters liste - accumulator pattern *)
2 fun rev_helper [] acc = acc
3   | rev_helper (h::t) acc = rev_helper t (h::acc);
4
5 fun reverse lst = rev_helper lst [];
6
7 reverse [1, 2, 3, 4, 5]; (* => [5, 4, 3, 2, 1] *)
```